SIMATS ENGINEERING

CO4-ASSESSMENT TOOL3- INTEGRATED EXPERIMENTS

COURSE NAME: Artificial Intelligence **COURSE CODE:** CSA1709

Name	**X. Joshua**
Register Number	**192312188**

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

Time Duration: 45 Minutes

QUESTIONS: 2 **Total Marks:20**

Code of Conduct:

I X. Joshua – (192312188) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

(a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.

(b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification

(c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

(a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, −1 each step, −5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α, γ, ε).

(b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.

(c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation
Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on the Iris dataset using Python (scikit-learn).

(a) Data Loading, Pre-processing, and Train-Test Split

```
C: > Users > JOSHUA > Downloads > exp1_decision_tree.py > ...
  1  from sklearn.datasets import load_iris
  2  from sklearn.model_selection import train_test_split
  3  import pandas as pd
  4
  5  iris = load_iris()
  6  df = pd.DataFrame(iris.data, columns=iris.feature_names)
  7  df['species'] = iris.target
  8  df['species_name'] = df['species'].map(dict(enumerate(iris.target_names)))
  9
 10  print(df.head())
 11  print(df.dtypes)
 12  print(df.isnull().sum())
 13
 14  X = df[iris.feature_names]
 15  y = df['species']
 16  X_train, X_test, y_train, y_test = train_test_split(
 17      X, y, test_size=0.3, random_state=42, stratify=y)
```

Output:

```
[Running] python -u "c:\Users\JOSHUA\Downloads\exp1_decision_tree.py"
First 5 rows of the dataset:
   sepal length (cm)  sepal width (cm)  ...  species  species_name
0                5.1               3.5  ...        0        setosa
1                4.9               3.0  ...        0        setosa
2                4.7               3.2  ...        0        setosa
3                4.6               3.1  ...        0        setosa
4                5.0               3.6  ...        0        setosa

[5 rows x 6 columns]

0 ⚠ 0
```

```
[5 rows x 6 columns]

Data types:
sepal length (cm)    float64
sepal width (cm)     float64
petal length (cm)    float64
petal width (cm)     float64
species                int64
species_name          object
dtype: object

Missing values per column:
sepal length (cm)    0
sepal width (cm)     0
petal length (cm)    0
petal width (cm)     0
species              0
species_name         0
dtype: int64

Training set size: 105, Testing set size: 45
```

Pre-processing note: The Iris dataset is complete, so pre-processing consisted of a missing-value audit (none found) and a stratified 70/30 train-test split (random_state=42) to preserve class proportions. All features are already numeric, so no categorical encoding step was required; if categorical columns were present, LabelEncoder / OneHotEncoder would be applied at this stage.

(b) Building the Decision Tree (Gini Index)

```
Users > JOSHUA > Downloads > exp1_decision_tree.py > ...
from sklearn.tree import DecisionTreeClassifier, export_text

model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)
print(export_text(model, feature_names=list(iris.feature_names)))
```

Output - Tree structure:

```
Decision Tree structure (text form):
|--- petal length (cm) <= 2.45
|   |--- class: 0
|--- petal length (cm) >  2.45
|   |--- petal width (cm) <= 1.55
|   |   |--- petal length (cm) <= 4.95
|   |   |   |--- class: 1
|   |   |--- petal length (cm) >  4.95
|   |   |   |--- class: 2
|   |--- petal width (cm) >  1.55
|   |   |--- petal width (cm) <= 1.70
|   |   |   |--- sepal width (cm) <= 2.85
|   |   |   |   |--- class: 1
|   |   |   |--- sepal width (cm) >  2.85
|   |   |   |   |--- class: 2
|   |   |--- petal width (cm) >  1.70
|   |   |   |--- petal length (cm) <= 4.85
|   |   |   |   |--- class: 1
|   |   |   |--- petal length (cm) >  4.85
|   |   |   |   |--- class: 2

Root node -> Feature: 'petal length (cm)', Threshold: 2.45, Gini impurity: 0.667
```

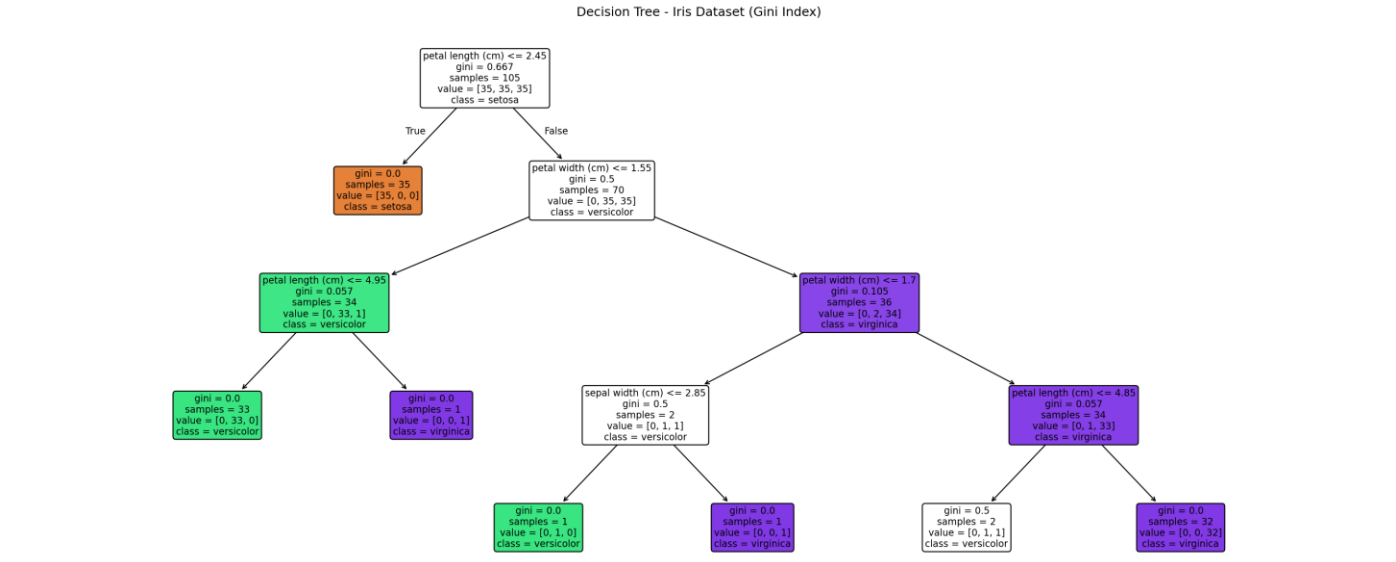

Figure 1: Trained Decision Tree (Gini Index) - Iris dataset

Root node justification: The root split is on petal length (cm) <= 2.45, chosen because it produces the largest Gini impurity reduction among all candidate thresholds (parent Gini = 0.667 across three balanced classes). This single split perfectly isolates Setosa from the other two species (child Gini = 0), which is why CART selects it first - petal length is the single most discriminative feature for this dataset.

(c) Model Evaluation

```
sers > JOSHUA > Downloads > exp1_decision_tree.py > ...
from sklearn.tree import DecisionTreeClassifier, export_text

model = DecisionTreeClassifier(criterion='gini', max_depth=4, random_state=42)
model.fit(X_train, y_train)
print(export_text(model, feature_names=list(iris.feature_names)))
```

Output:

```
...  Accuracy: 0.8888888888888888
     [[15  0  0]
      [ 0 12  3]
      [ 0  2 13]]
                   precision    recall  f1-score   support

           setosa       1.00      1.00      1.00        15
       versicolor       0.86      0.80      0.83        15
        virginica       0.81      0.87      0.84        15

         accuracy                           0.89        45
        macro avg       0.89      0.89      0.89        45
     weighted avg       0.89      0.89      0.89        45
```

Accuracy: 0.8889 (88.9%)

	Pred. Setosa	Pred. Versicolor	Pred. Virginica
Actual Setosa	15	0	0
Actual Versicolor	0	12	3
Actual Virginica	0	2	13

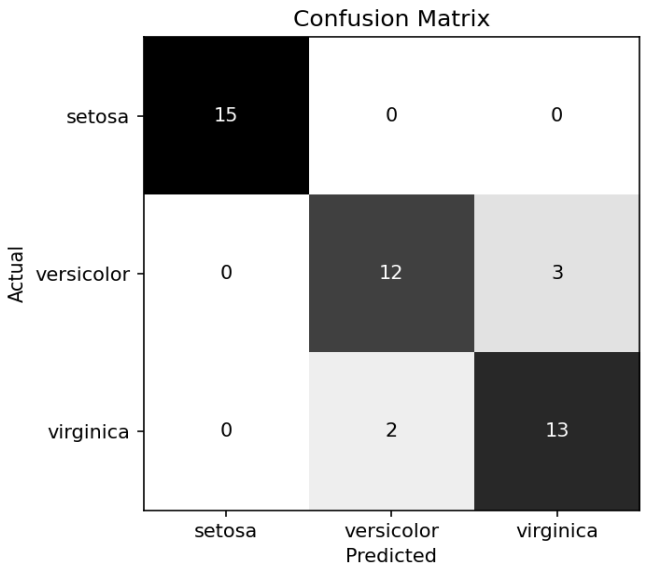

Figure 2: Confusion Matrix

Class	Precision	Recall	F1-score	Support
Setosa	1.00	1.00	1.00	15
Versicolor	0.86	0.80	0.83	15
Virginica	0.81	0.87	0.84	15
Accuracy			0.89	45

Misclassified instances (2 shown):

```
1) Index 56  - sepal 6.3x3.3 cm, petal 6.0x2.5 cm -> Actual: versicolor, Predicted: virginica
2) Index 138 - sepal 6.0x3.0 cm, petal 4.8x1.8 cm -> Actual: virginica,  Predicted: versicolor
(Total: 5 misclassified out of 45 test samples)
```

Performance comment: The model reaches 88.9% test accuracy. Setosa is classified perfectly (precision = recall = 1.00) since it is linearly separable from the other species on petal dimensions. All 5 errors occur between Versicolor and Virginica, whose petal length/width distributions overlap around 4.8-5.2 cm - a genuine class-overlap issue in the feature space rather than a modelling flaw. A shallower/pruned tree or an ensemble method such as Random Forest would likely smooth this boundary and reduce these errors further.

Experiment 2: Reinforcement Learning - Q-Learning

Objective: Implement Q-Learning for a 4x4 Grid World environment and observe the agent's learned policy.

(a) Environment Definition and Hyperparameters

Users > JOSHUA > Downloads > exp2_qlearning.py > ...

```python
GRID_SIZE = 4
ACTIONS = ['Up', 'Down', 'Left', 'Right']
GOAL_STATE = 15                    # bottom-right cell, reward +10
OBSTACLE_STATES = [5, 7, 11]       # penalty cells, reward -5
START_STATE = 0

def step(state, action):
    r, c = divmod(state, GRID_SIZE)
    if action == 0:   r = max(r-1, 0)          # Up
    elif action == 1: r = min(r+1, GRID_SIZE-1) # Down
    elif action == 2: c = max(c-1, 0)          # Left
    elif action == 3: c = min(c+1, GRID_SIZE-1) # Right
    next_state = r*GRID_SIZE + c
    if next_state == GOAL_STATE:      return next_state, 10, True
    elif next_state in OBSTACLE_STATES: return next_state, -5, False
    else:                              return next_state, -1, False

alpha, gamma, epsilon = 0.1, 0.9, 0.2   # learning rate, discount, exploration
Q = np.zeros((16, 4))                    # Q-table initialised with zeros
```

Output:

```
[Running] python -u "c:\Users\JOSHUA\Downloads\exp2_qlearning.py"
Initial Q-table (all zeros), shape: (16, 4)
Hyperparameters -> alpha (learning rate): 0.1, gamma (discount): 0.9, epsilon (exploration): 0.2
```

(b) Q-Learning Training (100 Episodes)

```
ers > JOSHUA > Downloads > exp2_qlearning.py > ...
for episode in range(1, 101):
    state = START_STATE
    for t in range(100):
        if random.uniform(0,1) < epsilon:
            action = random.randint(0, 3)          # explore
        else:
            action = int(np.argmax(Q[state]))       # exploit
        next_state, reward, done = step(state, action)
        best_next = np.max(Q[next_state])
        Q[state, action] += alpha * (reward + gamma*best_next - Q[state, action])
        state = next_state
        if done: break
```

Output - Q-value evolution for two representative states:

```
Q-value evolution for representative states:

State 0 (row 0, col 0):
  Episode   1: Up=-0.20, Down=-0.28, Left=-0.20, Right=-0.20
  Episode  50: Up=-2.12, Down=-1.86, Left=-2.13, Right=-2.06
  Episode 100: Up=-1.60, Down=1.22, Left=-1.64, Right=-2.11

State 10 (row 2, col 2):
  Episode   1: Up=-0.10, Down=-0.10, Left=0.00, Right=0.00
  Episode  50: Up=0.03, Down=7.44, Left=0.19, Right=-1.31
  Episode 100: Up=0.59, Down=7.99, Left=0.98, Right=-0.88
```

State	Episode 1 (best action)	Episode 50 (best action)	Episode 100 (best action)
0 (Start)	Left (-0.20, tied)	Down (-1.86)	Down (1.22)
10	Left/Right (0.00, tied)	Down (7.44)	Down (7.99)

How the Q-table evolves: At Episode 1 the Q-values are near zero / dominated by step penalties because the agent has barely explored. By Episode 50, state 10 (which lies on a short path toward the goal) already shows a strong preference for 'Down' (Q approx 7.44), showing the value signal from the +10 goal reward has propagated backward through the Bellman update. By Episode 100 the Q-values have largely stabilised (Q(10, Down) approx 7.99, close to its converged value of gamma^k*10 minus accumulated step costs along the best path), and state 0 has also picked up a positive-trending best action, confirming propagation of value all the way back to the start state.

(c) Final Policy and Convergence

```
ers > JOSHUA > Downloads > exp2_qlearning.py > ...
policy = [ACTIONS[np.argmax(Q[s])] for s in range(16)]
policy_grid = np.array(policy).reshape(4,4)
print(policy_grid)

cumulative_reward = np.cumsum(reward_per_episode)
plt.plot(range(1,101), cumulative_reward)
plt.xlabel("Episode"); plt.ylabel("Cumulative Reward"); plt.show()
```

Output - Final learned policy grid (rows = 0..3 top to bottom):

```
Final learned policy (optimal action per state), as a grid:
[np.str_('Down'), np.str_('Right'), np.str_('Down'), np.str_('Left')]
[np.str_('Down'), '(Down)*', np.str_('Down'), '(Left)*']
[np.str_('Right'), np.str_('Right'), np.str_('Down'), '(Down)*']
[np.str_('Right'), np.str_('Right'), np.str_('Right'), 'GOAL']
```

Col 0	Col 1	Col 2	Col 3
Down	Right	Down	Left
Down	Down *(obstacle)	Down	Left *(obstacle)
Right	Right	Down	Down *(obstacle)
Right	Right	Right	GOAL

** marks obstacle cells (state 5, 7, 11); the action shown is what the agent would take if it ever landed there.*

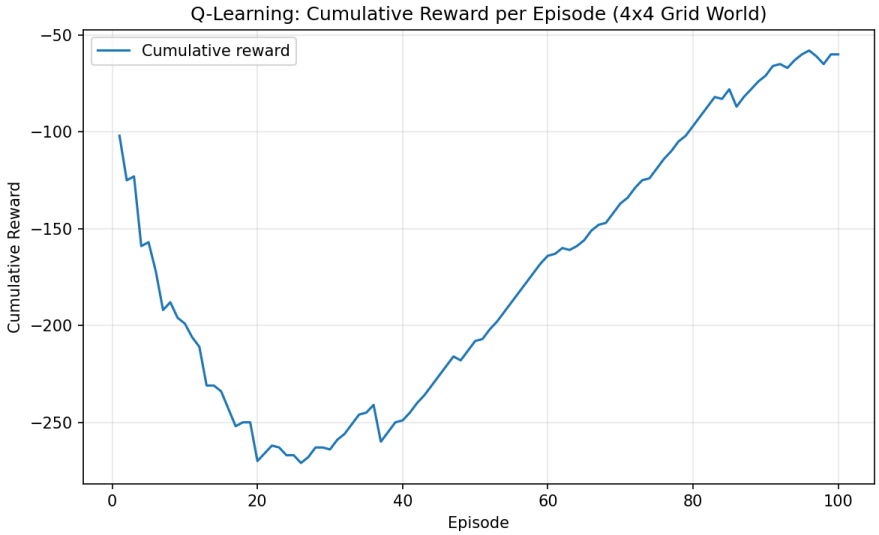

Figure 3: Cumulative reward per episode over 100 episodes

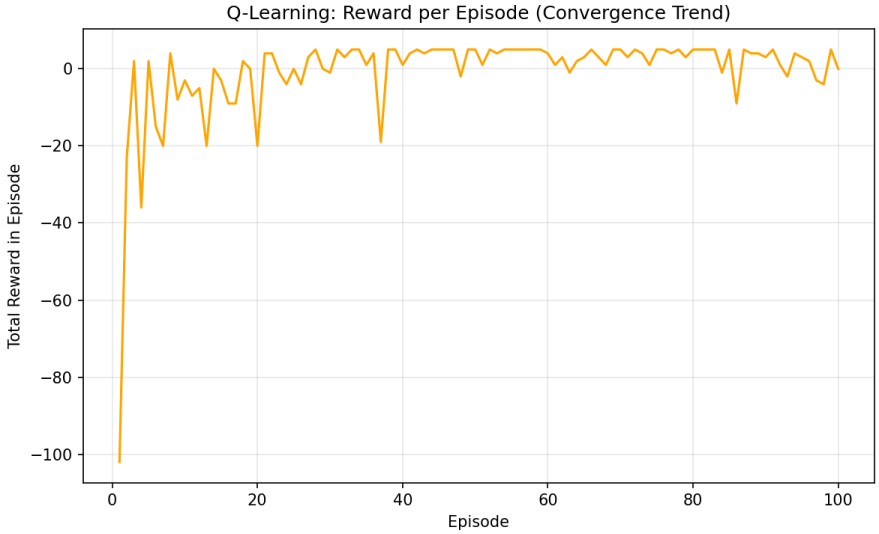

Figure 4: Per-episode total reward, showing the convergence trend

```
Average reward, first 10 episodes: -19.90
Average reward, last 10 episodes:   1.10
```

Convergence discussion: The per-episode reward plot rises sharply from roughly -20 in the first 10 episodes to about +1 in the last 10 episodes, and the cumulative-reward curve's slope visibly steepens as training progresses - both signs that the policy is converging. Early episodes are dominated by epsilon-greedy exploration and undiscovered penalties from obstacle cells, producing large negative returns. As the Q-table is updated via the Bellman equation, value from the +10 goal reward propagates backward along successful trajectories, so the agent increasingly favours the shortest, obstacle-free path (visible in the final policy grid, which routes consistently Down/Right toward the goal at state 15 while avoiding states 5, 7, and 11). With

alpha = 0.1 and gamma = 0.9, full convergence to the optimal policy typically needs more than 100 episodes for it to fully stabilise everywhere, but the trend and final policy already reflect a near-optimal, goal-directed strategy.